TD1 — Terminal & Git fundamentals

Git, Python Environments & Coding Best Practices

MSA-DATB07-03 — 1 h 30 practical

Learning outcome. Install the tools, move, search and read files from the shell, then initialise a Git repository, stage and commit changes, inspect the history with `git log` and `git diff`, and undo mistakes with `git restore` and `git revert`.

Tip — how to work

Keep a file `td1_answers.md` with, for every question, the command you typed *and* its answer. Exercise 3 tells you where to create it. Use `Tab` completion everywhere. The slides and the bash cheat-sheet are on the course GitHub page, https://github.com/msa-datb07-git: keep them open. Questions marked **(bonus)** are for those who finish early.

Part 0 — Install and set up *(20 min)*

Exercise 1 — Install Git, Python and VS Code

Follow the column of your system. If one of the tools is already installed, check its version and skip the step.

Windows

1. **VS Code**: https://code.visualstudio.com, default options.
2. **Git for Windows**: https://git-scm.com. Keep the defaults, except two screens: *default editor* → "Use Visual Studio Code"; *name of the initial branch* → "Override" and type `main`. Git for Windows installs Git *and* Git Bash, but **not** Python.
3. **Python 3.10 or newer**: https://www.python.org/downloads. On the first screen, tick **Add python.exe to PATH**.
4. In VS Code: Extensions (`Ctrl+Shift+X`) → install *Python* (Microsoft). Then `Ctrl+Shift+P` → *Terminal: Select Default Profile* → *Git Bash*.

macOS

1. Open *Terminal* (Spotlight: `Cmd+Space`, type "terminal"). Type `git --version`. If macOS offers to install the *command line developer tools*, accept: this installs Git.
2. **Python 3.10 or newer**: https://www.python.org/downloads, run the `.pkg` installer.
3. **VS Code**: https://code.visualstudio.com, drag it into *Applications*.
4. In VS Code: Extensions (`Cmd+Shift+X`) → install *Python* (Microsoft). Then `Cmd+Shift+P` → *Shell Command: Install 'code' command in PATH*.

Close VS Code completely, reopen it, and open a terminal with `Ctrl+‘` (the key left of 1). Check everything:

```
echo $0            # bash (Windows, Git Bash) or zsh / -zsh (macOS)
git --version      # git version 2.x
python --version   # Python 3.10 or newer   (macOS: python3 --version)
code --version     # the VS Code version
```

Pitfall — python opens the Microsoft Store, or says "not found"

Windows has a fake `python` that sends you to the Store. Fix: Windows *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, turn off the two *python* entries, then reopen VS Code. If it still fails, run the Python installer again and tick *Add python.exe to PATH*.

Exercise 2 — Tell Git who you are

Once per computer, type these four commands with your own name and e-mail (the e-mail you will use for GitHub in session 3):

```
git config --global user.name  "Firstname Lastname"
git config --global user.email "firstname.lastname@student.example.edu"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

- `git config` changes a Git setting; `--global` means "for every repository on this computer".
- `user.name` and `user.email` are written on every commit you make, so that your teammates know who did what.
- `init.defaultBranch main`: every new repository starts with one line of work, called a *branch*. This gives it the name `main`. Older Git versions call it `master`; with this setting everybody has the same name.
- `core.editor "code --wait"`: sometimes Git needs you to write a text, for example a message. It then opens VS Code (`code`) and waits (`--wait`) until you save and close the tab before continuing.

Check with `git config --global --list`: you should see the four values.

Exercise 3 — Your course folder, and the TD material

1. Go to your *Documents* folder and create a `cours` folder in it:

   ```
   cd ~/Documents          # ~ = your home folder
   mkdir -p cours          # create the folder (-p: no error if it already exists)
   cd cours
   pwd                     # check where you are
   ```

   If `cd ~/Documents` answers "No such file or directory" (common on Windows when Documents is in OneDrive), type `cd` followed by a space, drag your *Documents* folder from the file explorer onto the terminal, and press Enter.

2. Create your answers file *here*, in `cours`, next to the repositories and not inside them (Git would otherwise see it as a new file of the project): `code td1_answers.md`.

3. Download the TD material. `git clone <address>` downloads a copy of a project *with its whole history* and creates a folder with its name. The address of each repository is on the course GitHub page, https://github.com/msa-datb07-git (green *Code* button, *HTTPS* tab, copy). For this TD:

   ```
   git clone https://github.com/msa-datb07-git/td1-shell-and-git.git
   cd td1-shell-and-git
   ls
   ```

Pitfall — CRYPT_E_NO_REVOCATION_CHECK on Windows

Some networks (company, campus) block a certificate check that Windows performs. If `git clone` or `curl` fails with this message, run `git config --global http.schannelCheckRevoke false` for Git, and add `--ssl-no-revoke` to `curl` commands.

Part A — Shell treasure hunt *(30 min)*

Everything happens in `shell-lab/`: `cd shell-lab`. No file explorer, no mouse: terminal only.

Exercise 4 — Orientation

1. Print the absolute path of where you are.
2. List the content, then list it again showing hidden entries. What is hidden at the root of `shell-lab`?
3. Read `README.txt` without opening an editor.
4. How many files (not folders) does the whole tree contain? Hint: `find . -type f` lists them, `wc -l` counts lines.

Exercise 5 — Data

1. Show the first 3 lines of `data/raw/sales_2024_q1.csv`. What are the columns?
2. How many *data rows* (header excluded) does each quarterly file contain? One command for the four files at once (hint: `*.csv`).
3. Which of the four CSV files is the biggest? Hint: `ls -lS` sorts by size, biggest first.
4. How many rows of `sales_2024_q2.csv` concern the product `Gadget` in the `North` region? Hint: two `grep` chained with a pipe.

Exercise 6 — Search

1. One file in the tree contains the word `PENGUIN`. Which one? One command, from `shell-lab`.
2. List every Markdown file (`.md`) in the tree, then only those whose name contains `meeting`.
3. Do the meeting notes mention an API key? Print the matching lines with their file name and line number.

Exercise 7 — Logs and pipelines

1. How many lines of `logs/server.log` are at level `ERROR`?
2. Which error message is the most frequent, and how many times does it appear? Build the pipeline one step at a time, looking at the result after each step: `grep`, then `| cut -c27-`, then `| sort`, then `| uniq -c`, then `| sort -nr`.
3. Write the `ERROR` lines into a new file `logs/errors.txt`, then *add* the `WARN` lines at the end of the same file. How many lines does the result have?
4. Show the last 5 lines of `server.log`.

Exercise 8 — Environment

1. Print your home folder and your `PATH`. Which `python` runs when you type `python`?
2. Create a variable `PROJECT=sales` and print it. Open a *new* terminal (the `+` of the terminal panel): is it still there? Why?
3. **(bonus)** Who are you, on which machine, on which system? Three commands from the cheat-sheet.

Exercise 9 — Network and system

Write every command *for your system*, and next to it the name on the other system when it differs (slides and cheat-sheet). None of these commands sends anything of yours: they only test or read.

1. Test whether `github.com` answers, with `ping` and 4 test messages. Did all messages come back? What is the average time?
2. Follow the route to `github.com` (`tracert` or `traceroute`). How many steps? Do some steps show `* * *`? Does that mean the route is broken?
3. What is the IP address of `github.com`?
4. What status code does `https://github.com` return? (`curl -I`)
5. What is the current temperature in Paris? Use the address from the slides and display the answer with `python -m json.tool`.
6. **(bonus)** Which sub-folder of `shell-lab` takes the most space? Biggest last.
7. **(bonus)** List your running processes and find the PID of your shell (do not stop it).
8. **(bonus)** On macOS, `./scripts/report.sh` answers "Permission denied". Why? Fix it, run it. What does it print? (On Windows it runs directly: just run it.)
9. **(bonus)** Pack the `logs/` folder into `logs.tar.gz`, list what is inside without extracting it, then delete the archive.

Tip — clean up before Part B

Go back to the root of the repository (`cd ..`) and run `git status`: it shows the files you created or changed (`logs/errors.txt`…). Delete what you created with `rm`, cancel the other changes with `git restore .` (`.` = everything here), and check that `git status` says "working tree clean".

Part B — Your first repository *(35 min)*

Exercise 10 — Seven commits, one project

Leave the cloned repository (`cd ..`, you are back in `cours`) and build your own project from scratch.

1. Create a folder `my_sales`, go into it and turn it into a repository with `git init`. Check with `ls -la` and `git status`.
2. Create `README.md` containing one title line, for example `# My sales analysis`. Stage it with `git add`, look at `git status`, then commit it with a clear message.
3. Create a `data` folder and copy `../td1-shell-and-git/project/data/sales.csv` into it. Commit.
4. Create `analysis.py` with this function, then commit:

```python
import csv


def load_sales(path):
    """Read a sales CSV file and return a list of dictionaries (one per row)."""
    with open(path, newline="") as f:
        return list(csv.DictReader(f))
```

5. Add a function `total_revenue(rows)` that returns the sum of `float(r["quantity"]) * float(r["unit_price"])` over all rows. Before committing, run `git diff`, then `git add analysis.py`, then `git diff` again and `git diff --staged`. Explain in one sentence what each of the three outputs shows. Commit.
6. Create a `.gitignore` file containing the four lines `__pycache__/`, `.venv/`, `outputs/`, `.env`. Commit it.

7. Run `python -c "import analysis"`: it creates a `__pycache__` folder (`ls`). Does `git status` show it? Why?
8. Create `outputs/report.txt` with any content, and a file `.env` containing `API_KEY=secret`. Run `git status`: both must be invisible.
9. Change `total_revenue` so that it returns `0.0` when the list is empty. Commit with a message that starts with "Fix".
10. Show the history with one line per commit, then with the list of files changed in each commit.

Exercise 11 — Reading someone else's history

Back in `td1-shell-and-git` (`cd ../td1-shell-and-git`).

1. Show the history with one line per commit. How many commits? Who made the third one from the top? (Hint: `git log` without options shows the authors.)
2. One commit added a file that should never have been committed. Find it with `git log --stat`, then show its content with `git show` and the commit identifier.
3. What did the commit "Fix total_revenue crash on empty CSV" change? Show it with `git show`.
4. **(bonus)** Compare `project/analysis.py` between the commit that created it and today: `git diff <id> HEAD -- project/analysis.py`. How many functions were added?

Exercise 12 — Undo

Still in `td1-shell-and-git`. After each scenario, `git status` must say "working tree clean".

1. **Staged too early.** Add a line to `README.md`, stage it, then change your mind: unstage it *without losing the line*. Then throw the line away.
2. **A bad commit: revert it.** Cancel the commit that added `config.ini` with `git revert`. VS Code opens with a message: keep it, save, close the tab. What does `git log --oneline` show now? Is `config.ini` still in the folder? Is it still in the history?
3. **The exception: a commit you just made.** Add a line to `README.md` and commit it with the message "oops". Undo this last commit with `git reset --soft HEAD~1`, check with `git status` that your change is still staged, then commit it again with a proper message.
4. In one sentence: why is `git revert` the default way to undo a commit, and `git reset` only an exception?

Part C — Checklist *(5 min)*

1. In Git Bash, what does `echo $0` print? What does it mean if it prints something else?
2. What does `Ctrl+C` do in a terminal? How do you copy text instead?
3. A file appears in `git status` although its name is in `.gitignore`. Why?
4. Show your `my_sales` history to the instructor: `git log --oneline --stat`.

Deliverable

Before leaving: `td1_answers.md` filled in, in your `cours` folder; the `my_sales` repository with at least 6 commits and a clean working tree; the three undo scenarios done in `td1-shell-and-git`. Nothing to send today: session 3 puts your work on GitHub.